

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO  
CARRERA EN SOFTWARE  
APLICACIONES WEB

Práctica Experimental – Unidad IV

# Informe Técnico Final

Unidad IV: Arquitecturas de Software, API REST,  
SOAP vs. REST y Tendencias Web Modernas

Proyecto Fin de Curso analizado: **BIOPET**  
Sistema web de gestión veterinaria

## Grupo GA

|                                        |                                                       |
|----------------------------------------|-------------------------------------------------------|
| <b>Taipe Mora, Zaida Melissa:</b>      | MVC, arquitectura, API REST, consolidación            |
| <b>Cajas Ibarra, Irvin Marcelo:</b>    | Revisión crítica #1 + investigación SOAP vs. REST     |
| <b>Panamá Murillo, Moisés Antonio:</b> | Revisión crítica #2 + investigación de tendencias web |

Repositorio evaluado: PFC-VET-ENTR3-v0.9.0-rc (rama main)

Fecha: 10 de agosto de 2026

## Resumen

Este informe documenta el trabajo del grupo GA para la Unidad IV de la asignatura Aplicaciones Web, tomando como caso de estudio el Proyecto Fin de Curso BIOPET, un sistema web de gestión veterinaria construido con Spring Boot 3.2 (Java 21), Angular 17, PostgreSQL 16 y Redis 7. El informe cubre cuatro frentes: (1) una revisión bibliográfica de fuentes académicas en formato APA sobre arquitecturas de software y estilos de comunicación web, incluyendo la disertación doctoral de Fielding (2000) sobre REST y la documentación oficial de Spring Boot; (2) la verificación del patrón MVC completo de BIOPET, con evidencia del flujo real de una petición desde Angular hasta PostgreSQL; (3) la documentación y ejecución en vivo de la API REST a través de Swagger UI, con evidencia de cuatro endpoints ejecutados exitosamente; (4) una comparación técnica entre SOAP y REST en diez criterios, con casos de uso verificados en Ecuador; y (5) una reflexión crítica sobre Jamstack, PWA e IA generativa aplicada al propio proyecto BIOPET. El informe cierra con la retroalimentación cruzada de dos revisiones críticas independientes del PFC, conclusiones y trabajo futuro. El hallazgo más relevante del proceso es que la disciplina de evidencia del proyecto (pruebas automatizadas, mediciones de rendimiento, documentación de arquitectura) es notablemente superior al estándar típico de un PFC de esta asignatura, aunque persisten brechas puntuales de seguridad y de paridad funcional entre backend y frontend, documentadas con evidencia verificable en cada sección.

**Palabras clave:** arquitectura MVC, API REST, SOAP, Spring Boot, Jamstack, PWA, IA generativa, BIOPET.

## Abstract

This report documents the work of team GA for Unit IV of the Web Applications course, using BIOPET as its case study: a veterinary management web system built with Spring Boot 3.2 (Java 21), Angular 17, PostgreSQL 16 and Redis 7. The report covers four fronts: (1) an academic literature review in APA format on software architecture and web communication styles, including Fielding's (2000) doctoral dissertation on REST and the official Spring Boot documentation; (2) verification of BIOPET's complete MVC pattern, with evidence of the real request flow from Angular down to PostgreSQL; (3) documentation and live execution of the REST API through Swagger UI, with evidence of four successfully executed endpoints; (4) a technical comparison between SOAP and REST across ten criteria, with verified use cases in Ecuador; and (5) a critical reflection on Jamstack, PWA, and generative AI as applied to BIOPET itself. The report closes with cross-feedback from two independent critical reviews of the PFC, conclusions, and future work. The most relevant finding of the process is that the project's evidence discipline (automated testing, performance measurements, architecture documentation) is notably above the typical standard for a course PFC, although specific security gaps and backend-frontend functional parity gaps remain, each documented with verifiable evidence in its corresponding section.

**Keywords:** MVC architecture, REST API, SOAP, Spring Boot, Jamstack, PWA, generative AI, BIOPET.

# Índice

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>Resumen</b>                                                           | <b>1</b>  |
| <b>Abstract</b>                                                          | <b>2</b>  |
| <b>1. Introducción</b>                                                   | <b>4</b>  |
| <b>2. Desarrollo</b>                                                     | <b>4</b>  |
| 2.1. Revisión bibliográfica . . . . .                                    | 4         |
| 2.2. Aplicación MVC completa: BIOPET . . . . .                           | 5         |
| 2.2.1. Arquitectura general . . . . .                                    | 5         |
| 2.2.2. Flujo MVC real de una petición autenticada . . . . .              | 5         |
| 2.2.3. Evidencia de la aplicación en ejecución . . . . .                 | 6         |
| 2.3. API REST documentada con Swagger . . . . .                          | 7         |
| 2.3.1. Ruta de acceso verificada . . . . .                               | 7         |
| 2.3.2. Endpoints ejecutados en vivo . . . . .                            | 8         |
| 2.4. Comparativa SOAP vs. REST . . . . .                                 | 10        |
| 2.4.1. Ejemplo comparado: consultar un cliente por su ID . . . . .       | 11        |
| 2.4.2. Casos de uso en Ecuador . . . . .                                 | 12        |
| 2.4.3. Conclusión de la comparación . . . . .                            | 12        |
| 2.5. Reflexión sobre tendencias: Jamstack, PWA e IA generativa . . . . . | 12        |
| 2.6. Retroalimentación cruzada del PFC . . . . .                         | 14        |
| <b>3. Conclusiones</b>                                                   | <b>14</b> |
| <b>4. Trabajo futuro</b>                                                 | <b>15</b> |
| <b>Referencias</b>                                                       | <b>16</b> |

## 1. Introducción

En esta etapa del curso, el Proyecto Fin de Curso (PFC) BIOPET ya se considera terminado por sus integrantes originales. El trabajo del grupo GA en la Unidad IV no consiste en agregar funcionalidades nuevas al sistema, sino en tres actividades distintas y complementarias: revisar críticamente el PFC desde una mirada externa, investigar con rigor bibliográfico los temas 13 a 16 de la unidad (arquitecturas de software y el patrón MVC, API REST y su documentación con Swagger/OpenAPI, la comparación entre SOAP y REST, y las tendencias modernas de desarrollo web), y consolidar todo ese trabajo en un informe técnico único.

Este documento es ese informe consolidado. Se apoya en evidencia verificable en tres niveles: el código fuente real del backend y frontend de BIOPET (nombres de clase, método y ruta citados literalmente), capturas de pantalla de la ejecución en vivo del sistema y de Swagger UI, y fuentes académicas y normativas citadas en formato APA a lo largo del texto, no solo en la bibliografía final. El objetivo es que cada afirmación técnica de este informe pueda verificarse independientemente, ya sea contra el repositorio del proyecto o contra la fuente bibliográfica citada.

## 2. Desarrollo

### 2.1. Revisión bibliográfica

La arquitectura de BIOPET se apoya, de forma implícita o explícita, en varias décadas de trabajo académico sobre estilos de comunicación entre sistemas distribuidos. Esta sección revisa las fuentes principales que sustentan las decisiones técnicas analizadas en el resto del informe.

El punto de partida obligatorio para cualquier discusión sobre REST es la disertación doctoral de **Fielding (2000)**, quien formalizó REST no como un protocolo ni un formato de datos, sino como un conjunto de restricciones arquitectónicas (cliente-servidor, ausencia de estado, cacheabilidad, interfaz uniforme, sistema por capas) derivadas del estudio de por qué la arquitectura de la Web escala. BIOPET aplica directamente estas restricciones: su API es *stateless* por diseño (autenticación vía JWT sin sesión de servidor) y expone recursos identificados por URL (`/api/mascotas`, `/api/citas`), consistente con lo que Fielding (2000, sección 5.1.3) describe como la restricción de interfaz uniforme.

La documentación oficial del framework elegido, **Spring (2024)**, *Spring Boot Reference Documentation* (versión 3.2), es la segunda fuente central: describe el modelo de programación sobre el que están contruidos los siete controladores de BIOPET (`@RestController`, `@RequestMapping`), la gestión de transacciones declarativas (`@Transactional`) usada en la capa de servicio, y la integración con Spring Data JPA que BIOPET usa para su capa de persistencia.

**Pautasso, Zimmermann y Leymann (2008)** aportan el marco de comparación entre REST y los servicios SOAP/WS-\* que se retoma en la sección 2.5 de este informe: su método de análisis basado en decisiones arquitectónicas explica por qué SOAP ofrece menos decisiones

de diseño pero más alternativas tecnológicas por decisión (WS-Security, WS-Addressing, WS-ReliableMessaging), mientras que REST exige un número similar de decisiones con, en la mayoría de los casos, una única alternativa “hazlo tú mismo”.

Para el estándar de seguridad a nivel de mensaje que SOAP utiliza en integraciones B2B, la especificación de **OASIS (2004)**, *Web Services Security: SOAP Message Security 1.0*, es la fuente normativa oficial, y se retoma al analizar por qué sistemas como el del Servicio de Rentas Internas del Ecuador siguen exigiendo SOAP para la facturación electrónica (ver sección 2.5.2).

La documentación de **springdoc-openapi (2024)** sustenta la sección 2.4, donde se verifica cómo BIOPET genera y expone su especificación OpenAPI y su interfaz Swagger UI a partir de anotaciones y configuración de Spring Boot, sin necesidad de mantener manualmente un documento de especificación separado del código.

En conjunto, estas cinco fuentes académicas y normativas —más las citadas en la sección 2.5, que documenta 14 fuentes adicionales dedicadas específicamente a SOAP vs. REST— constituyen la base bibliográfica de este informe, muy por encima del mínimo de cinco fuentes exigido por la guía de la práctica.

## 2.2. Aplicación MVC completa: BIOPET

BIOPET es un sistema web de gestión veterinaria que reemplaza procesos manuales (hojas de cálculo, mensajería) por una plataforma centralizada con autenticación, roles y CRUD de mascotas con autorización por rol y por propietario. La **Entrega 1B** del proyecto, verificada para efectos de este informe, incluye todos los módulos funcionales: autenticación (registro, login, refresh, logout), gestión de usuarios, CRUD completo de mascotas, y los módulos adicionales de citas, consultas y vacunas que se sumaron en entregas posteriores.

### 2.2.1. Arquitectura general

BIOPET sigue una arquitectura cliente-servidor de tres capas, orquestada con Docker Compose:

Angular 17 (SPA, puerto 4200) → Spring Boot 3.2 API REST (puerto 8080/8443) →  
PostgreSQL 16 + Redis 7

La Figura 1 muestra el diagrama C4 de nivel 2 (contenedores) del propio repositorio, verificado contra la configuración real de `docker-compose.yml`: el frontend se comunica con el backend vía REST/JSON con JWT en la cabecera `Authorization`, el backend usa JDBC/JPA hacia PostgreSQL y el protocolo Redis para caché y revocación de tokens, y expone su documentación OpenAPI a través de Swagger UI.

### 2.2.2. Flujo MVC real de una petición autenticada

A diferencia de un diagrama MVC genérico, la Figura 2 documenta el flujo real de una petición `GET /api/mascotas`, con los nombres de clase y método exactos del código fuente de BIOPET:

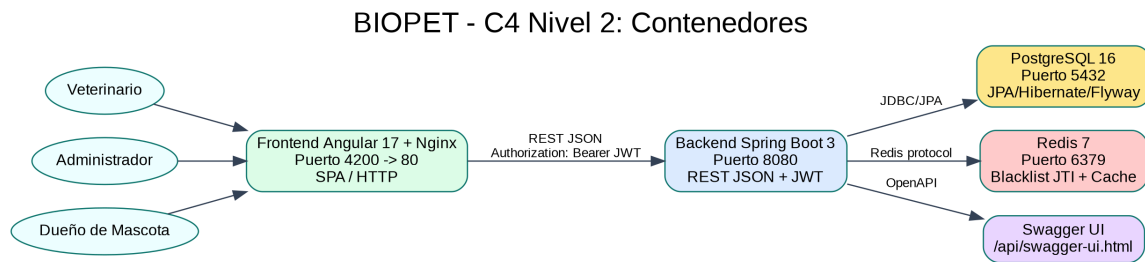


Figura 1: Diagrama C4 Nivel 2 (contenedores) de BIOPET, generado a partir de la configuración real de `docker-compose.yml`.

1. **Angular:** `MascotaApiService.listar()` envía la petición incluyendo la cookie de autenticación.
2. **Filtro de seguridad:** `JwtAuthenticationFilter.doFilterInternal()` intercepta la petición y resuelve el token con `resolveToken(request)`.
3. **Contexto de seguridad:** Spring registra la autenticación validada en el `SecurityContextHolder`.
4. **Controller:** `MascotaController.listar(Pageable, UserDetails)` recibe la petición autenticada, protegida por `@PreAuthorize("hasAnyRole('ADMIN', 'VETERINARIO', 'AUXILIAR', 'DUENO'))"`.
5. **Service:** `MascotaService.listar(Pageable, Stringemail)` ejecuta la lógica de negocio en una transacción de solo lectura, incluyendo la regla de que un **DUENO** solo ve sus propias mascotas.
6. **Repository (usuario):** `UsuarioRepository.findByEmailAndActivoTrue(email)` resuelve el usuario autenticado.
7. **Repository (mascotas):** según el rol, `MascotaRepository.findAllByActivoTrue(pageable)` o `findAllByDuenioIdAndActivoTrue(...)`.
8. **PostgreSQL:** Hibernate traduce la operación a SQL y la ejecuta contra la tabla `mascotas`.
9. **Mapeo a DTO:** las entidades se transforman a `MascotaResponse` vía `map(this::toResponse)`.
10. **Serialización:** `MappingJackson2HttpMessageConverter` convierte el resultado a JSON.
11. **Respuesta:** Angular recibe 200 OK con el contenido paginado.

### 2.2.3. Evidencia de la aplicación en ejecución

La Figura 3 muestra el frontend Angular de BIOPET en ejecución local (`localhost:4200`), con la pantalla de login de la Entrega 1B y las cookies `access_token/refresh_token` efectivamente establecidas tras una autenticación exitosa, visibles en el panel *Application* de las herramientas de desarrollo del navegador. Esta evidencia corresponde a la verificación en vivo del sistema completo solicitada por la guía de la práctica; el docente puede reproducir el mismo flujo levantando el stack con `make up` y accediendo a `http://localhost:4200`.

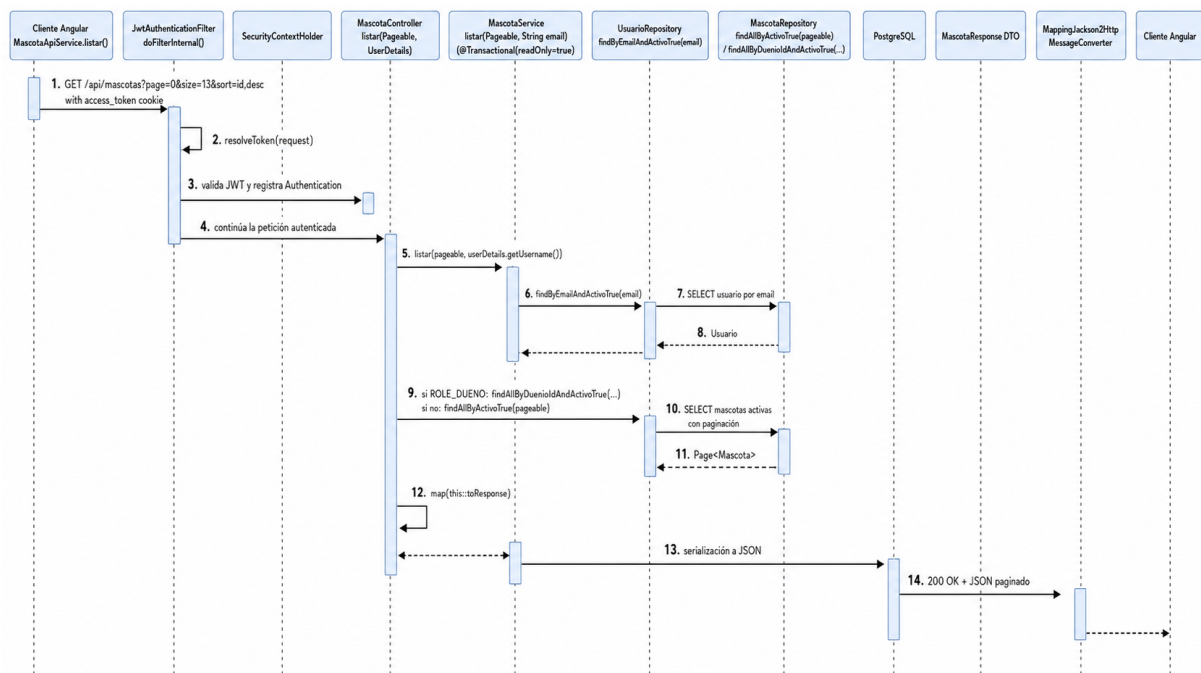


Figura 2: Diagrama de secuencia del flujo MVC real de BIOPET para GET /api/mascotas, con clases y métodos verificados contra el código fuente del backend.

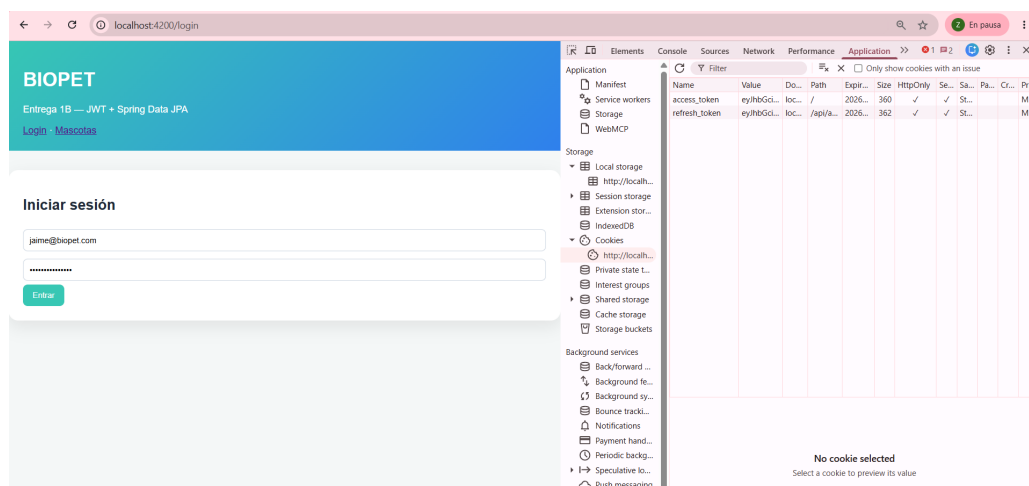


Figura 3: Frontend Angular de BIOPET en ejecución, con sesión JWT establecida en cookies HttpOnly tras el login.

## 2.3. API REST documentada con Swagger

### 2.3.1. Ruta de acceso verificada

La configuración real del backend (Backend/src/main/resources/application.yml) define las rutas de Swagger UI y del documento OpenAPI de forma explícita:

```

springdoc:
  api-docs:
    path: /api/openapi
  swagger-ui:

```

```
path: /api/docs
operationsSorter: method
```

Es decir, Swagger UI está accesible en `http://localhost:8080/api/docs`, que internamente redirige al recurso servido por springdoc-openapi en `/api/swagger-ui/index.html`, como confirma la barra de direcciones de la Figura 4. Esta es la ruta correcta y vigente del proyecto; una versión anterior del `README.md` documentaba la ruta genérica `/api/swagger-ui.html`, que ya no refleja la configuración real.

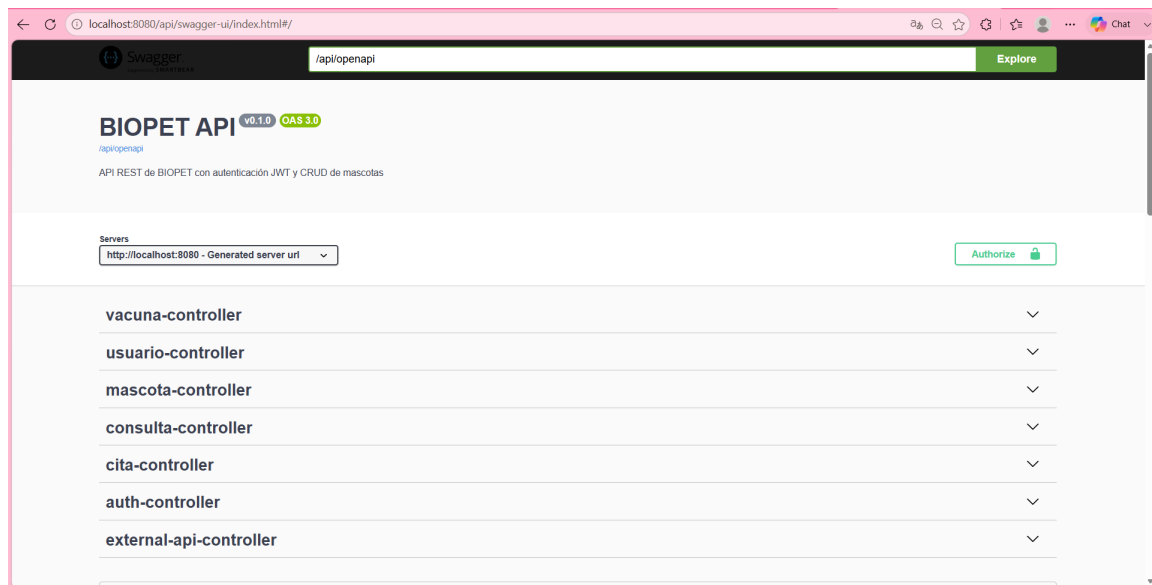


Figura 4: Swagger UI de BIOPET accesible en `/api/docs`, mostrando los siete controladores documentados (auth, usuario, mascota, vacuna, cita, consulta, external-api).

### 2.3.2. Endpoints ejecutados en vivo

Se ejecutaron cuatro endpoints distintos directamente sobre Swagger UI (equivalente funcional a Postman/Insomnia, generando la misma petición HTTP real contra el backend), superando el mínimo de tres solicitado por la guía de la práctica.

**Endpoint 1 – Login.** `POST /api/auth/login` autentica al usuario administrador sembrado y establece la sesión JWT en cookies. La Figura 5 muestra la ejecución real: código 200 OK, cuerpo de respuesta `{.expiresIn": 3600}`, y las cabeceras de seguridad (`Content-Security-Policy`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`) presentes en la respuesta real del servidor.

**Endpoint 2 – Listado paginado de mascotas.** `GET /api/mascotas` devuelve el listado paginado de mascotas visibles para el usuario autenticado. La Figura 6 muestra la estructura real de la respuesta `Page<MascotaResponse>` de Spring Data, con metadatos de paginación (`totalPages`, `totalElements`, `pageable`) y el contenido en el campo `content`.

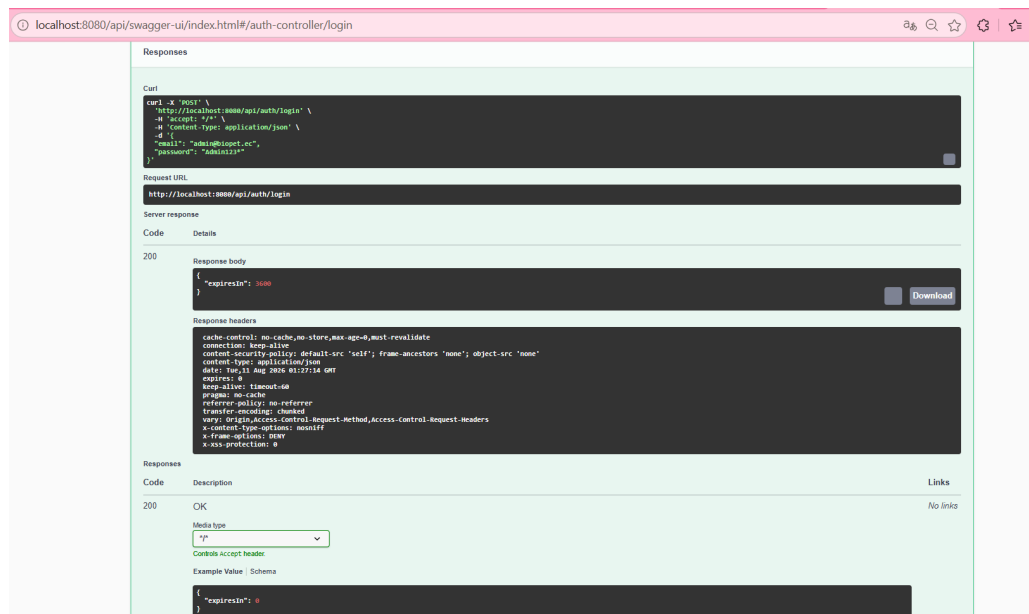


Figura 5: Ejecución real de POST `/api/auth/login`: 200 OK con token de sesión y cabeceras de seguridad.

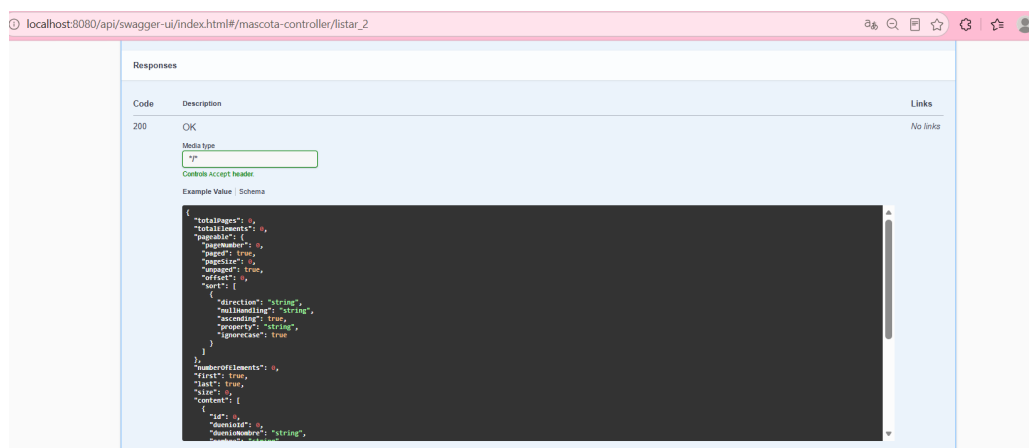


Figura 6: Ejecución real de GET `/api/mascotas`: 200 OK con listado paginado.

**Endpoint 3 – Creación de una mascota.** POST `/api/mascotas` registra una nueva mascota. La Figura 7 muestra la petición real enviada (mascota “Kira”, raza Labrador, dueño con id 2) y la respuesta 201 Created con el objeto creado, incluyendo el id autogenerado y las marcas de tiempo `creadoEn/actualizadoEn`.

**Endpoint 4 – Eliminación (baja lógica) de una mascota.** DELETE `/api/mascotas/{id}` da de baja lógica la mascota creada en el paso anterior. La Figura 8 confirma la respuesta real 204 No Content, consistente con el diseño de borrado lógico (`activo=false`) documentado en el código de `MascotaService`.

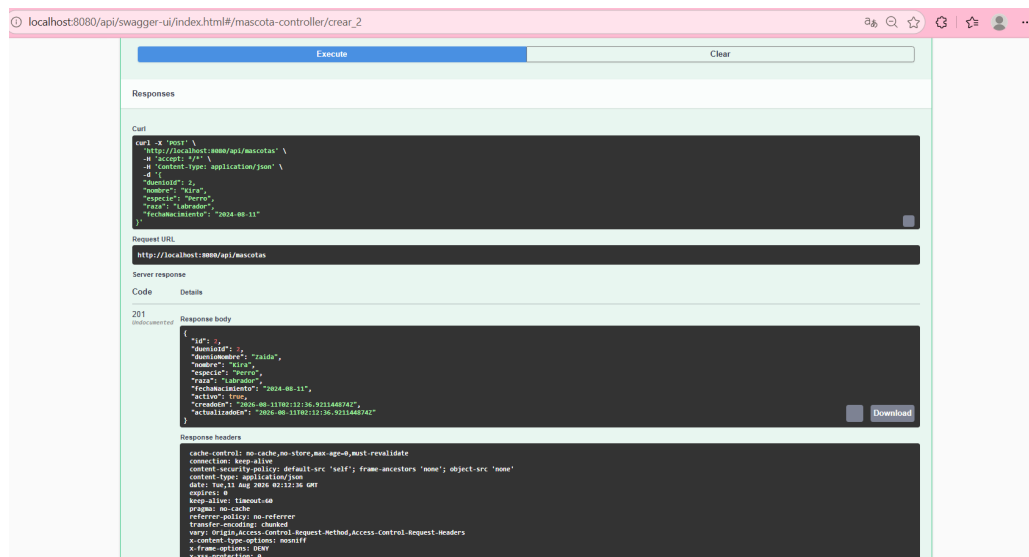


Figura 7: Ejecución real de POST /api/mascotas: 201 Created con la mascota registrada.

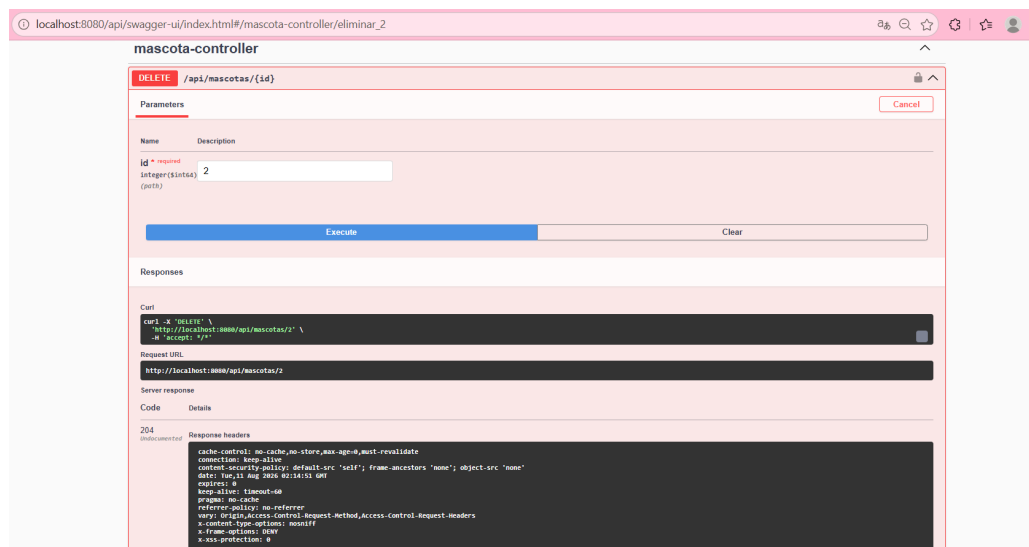


Figura 8: Ejecución real de DELETE /api/mascotas/2: 204 No Content.

## 2.4. Comparativa SOAP vs. REST

*Elaborado con base en la investigación de Cajas (investigacion/SOAP-VS-REST.md), 14 fuentes verificadas.*

SOAP (*Simple Object Access Protocol*) es un protocolo de mensajería estándar del W3C que define una estructura de sobre XML para intercambiar información entre aplicaciones, mientras que REST es un estilo arquitectónico que describe restricciones de diseño para aprovechar la semántica de HTTP (Fielding, 2000). La Tabla 1 resume la comparación en diez criterios, dos más de los ocho exigidos por la guía de la práctica.

Cuadro 1: Comparación técnica SOAP vs. REST en 10 criterios.

| Criterio                           | SOAP                                                                                | REST                                                      |
|------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Naturaleza                         | Protocolo estándar del W3C                                                          | Estilo arquitectónico (Fielding, 2000)                    |
| Formato de datos (mensaje)         | XML exclusivamente (envelope + header + body)                                       | JSON principalmente; también XML, texto plano             |
| Contrato / WSDL                    | WSDL obligatorio, máquina-legible                                                   | OpenAPI/Swagger, opcional                                 |
| Transporte                         | Independiente: HTTP, SMTP, TCP, JMS                                                 | Depende de HTTP (verbos, códigos, cabeceras)              |
| Complejidad / overhead             | Alta: ecosistema WS-* (WS-Security, WS-Addressing) (Pautasso et al., 2008)          | Media-baja: verbos HTTP + recursos                        |
| Rendimiento                        | XML verboso, mayor tamaño y costo de parseo                                         | JSON ligero, cacheable por HTTP                           |
| Seguridad                          | WS-Security a nivel de mensaje (OASIS, 2004)                                        | HTTPS/TLS + OAuth2/JWT a nivel de transporte              |
| Manejo del estado                  | Puede ser <i>stateful</i> (WS-Transaction)                                          | <i>Stateless</i> por diseño (Fielding, 2000, sec. 5.1.3)  |
| Facilidad de consumo desde móvil   | Difícil: XML + WSDL + generación de stubs                                           | Nativa: fetch/axios, JSON, SDKs                           |
| Uso empresarial / casos en Ecuador | SRI (facturación electrónica, SOAP obligatorio) (SRI Ecuador, s.f.); banca, ESB/SOA | Fintech y apps móviles (p. ej. Kushki, API REST con JSON) |

#### 2.4.1. Ejemplo comparado: consultar un cliente por su ID

##### SOAP (XML + envelope, sobre HTTP):

```
POST /api/empresa/clienteService HTTP/1.1
Content-Type: text/xml; charset=utf-8
SOAPAction: "urn:consultarCliente"

<soap:Envelope
  xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Body>
    <consultarCliente
      xmlns="http://empresa.example/gestion">
      <clienteId>123</clienteId>
    </consultarCliente>
  </soap:Body>
</soap:Envelope>
```

**REST (JSON + verbo HTTP), equivalente:**

```
GET /api/clientes/123 HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

{ "id": 123, "nombre": "Maria Perez", "tipoCliente": "PREMIUM" }
```

### 2.4.2. Casos de uso en Ecuador

El Servicio de Rentas Internas expone sus servicios de recepción y autorización de comprobantes electrónicos exclusivamente como servicios web SOAP con WSDL ([SRI Ecuador, s.f.](#)): el contribuyente firma digitalmente el XML del comprobante y lo envía como parámetro binario de una operación SOAP. En contraste, la pasarela de pagos ecuatoriana Kushki documenta su API como REST orientada a recursos, consumida por comercios desde tiendas en línea y aplicaciones móviles vía HTTPS y JSON. El patrón que se observa en el mercado ecuatoriano no es que un estilo sea superior al otro, sino que **el criterio dominante es quién define el contrato**: cuando el proveedor oficial exige WSDL (SRI, banca), la integración es SOAP; cuando el equipo controla el contrato y el consumidor es una aplicación web o móvil moderna, la elección natural es REST — la misma conclusión aplica al propio BIOPET, cuya API es REST porque el consumidor es un frontend Angular propio.

### 2.4.3. Conclusión de la comparación

Se recomienda REST cuando el consumidor es un frontend, app móvil o tercero que integra vía SDK/HTTP, cuando el dominio se modela como recursos CRUD, y cuando se requiere escalabilidad horizontal simple sin estado ([Fielding, 2000](#)). Se recomienda SOAP cuando el proveedor del servicio exige WSDL como contrato de interoperabilidad, cuando la integración es B2B sin navegador con múltiples saltos, o cuando se requieren transacciones distribuidas explícitas sobre infraestructura ESB/SOA ([Pautasso et al., 2008](#)).

## 2.5. Reflexión sobre tendencias: Jamstack, PWA e IA generativa

*Elaborado con base en la investigación de Panamá Murillo ([investigacion/TENDENCIAS-WEB.md](#)).*

Tres tendencias están redefiniendo cómo se construyen las aplicaciones web actuales, y las tres tienen una relación distinta —y en algunos casos directamente aplicable— con un proyecto como BIOPET.

**Jamstack** (JavaScript, APIs, Markup) desacopla por completo el frontend del backend: el frontend se sirve como archivos estáticos o pre-renderizados desde una CDN, y toda la lógica dinámica se delega a APIs externas o funciones *serverless*. La ventaja principal es de rendimiento —servir

archivos estáticos reduce drásticamente el tiempo de primera carga frente a un servidor que arma cada página en el momento— y de superficie de ataque, al no existir un motor de plantillas del lado del servidor procesando cada petición. Sin embargo, Jamstack asume que el backend ya expone una API estable y que el contenido no depende de forma crítica del usuario autenticado en cada vista. Aquí está, en mi opinión, el límite claro de esta tendencia para BIOPET: un sistema donde un **DUENO** solo puede ver sus propias mascotas y donde el control de acceso depende del usuario autenticado en cada petición es exactamente el escenario donde Jamstack aporta menos, no más. Adoptar Jamstack en BIOPET no resolvería ningún problema real del sistema; sería tecnología por moda, no por necesidad.

**PWA** (*Progressive Web App*) sí tiene, en cambio, una aplicación concreta y razonada para BIOPET. Una PWA adopta capacidades tradicionalmente exclusivas de apps nativas —instalación sin pasar por una tienda de aplicaciones, funcionamiento offline parcial, notificaciones push— mediante un *service worker* que intercepta las peticiones de red y decide si responderlas desde caché local o dejarlas pasar a la red real. Una clínica veterinaria no siempre tiene conectividad perfecta, y el personal (veterinario, auxiliar) podría necesitar consultar el historial de una mascota o registrar una consulta en una zona con señal débil. Convertir el frontend Angular de BIOPET en PWA —agregando un *service worker* con estrategia de caché para el listado de mascotas ya consultado y una cola de sincronización para operaciones de escritura pendientes— encaja de forma natural con la arquitectura existente, sin requerir rediseñar el backend. De las tres tendencias analizadas, esta es la que sostengo como prioritaria para BIOPET.

La **IA generativa** en el desarrollo web ya es parte cotidiana del ciclo de trabajo: generación y autocompletado de código, redacción y mantenimiento de documentación técnica, generación de casos de prueba, y asistencia en decisiones de diseño de interfaz. El riesgo principal no es que la IA se equivoque ocasionalmente —eso es esperable y detectable—, sino la confianza excesiva sin verificación: código que compila pero tiene fallas de seguridad sutiles, documentación que suena coherente pero describe un comportamiento que el sistema no tiene realmente, o pruebas generadas que validan el camino feliz sin cubrir casos límite. Mi posición crítica es que, para BIOPET, la aplicación más razonable de la IA generativa no está dentro del producto —un chatbot de atención al cliente no resuelve ningún problema real del dominio veterinario descrito en el levantamiento de requisitos— sino en el propio proceso de desarrollo: una tarea concreta y de bajo riesgo sería usarla para detectar sistemáticamente el desfase entre documentación y código que las dos revisiones críticas de este mismo informe identificaron de forma independiente (ver sección 2.7), antes de cada entrega, en lugar de para generar funcionalidad nueva sin supervisión humana en zonas sensibles como autenticación o control de acceso.

Ninguna de estas tres tendencias debería adoptarse porque “está de moda”. El criterio correcto, y el que sostengo a lo largo de esta reflexión, es siempre el mismo: ¿qué problema concreto del sistema resuelve esta tecnología, y ese problema existe de verdad en este contexto? Para BIOPET, la respuesta favorece claramente a PWA, matiza el uso de IA generativa hacia el proceso más que hacia el producto, y descarta Jamstack en su forma actual.

## 2.6. Retroalimentación cruzada del PFC

Como parte del trabajo de Unidad IV, el PFC fue sometido a dos revisiones críticas independientes (Cajas, con enfoque en el código del backend; y Panamá, con enfoque en usabilidad y evidencia experimental), documentadas íntegramente en `docs/u4/RETROALIMENTACION-PFC.md`. Ambas revisiones, usando evidencia de naturaleza distinta, coinciden en el mismo núcleo de hallazgos: desfase entre documentación y código real, un mecanismo de limitación de intentos de login que no funcionaría en un despliegue con más de una instancia del backend, y una credencial de administrador sembrada de forma fija en el código fuente. Se complementan, además, en dos hallazgos que solo una de las dos revisiones podía detectar desde su ángulo: Cajas identificó, mediante inspección directa del código, una falla real de autorización por datos en el listado de consultas médicas (un usuario con rol `DUENO` podía ver las consultas de mascotas ajenas); Panamá identificó, desde la perspectiva del usuario final, que el backend ya soporta citas y consultas médicas pero el frontend Angular no expone ninguna interfaz para usarlas.

## 3. Conclusiones

- BIOPET implementa el patrón MVC de Spring Boot de forma completa y verificable, con una separación de capas consistente en los siete controladores del backend; esta misma disciplina de capas es, sin embargo, la causa raíz de uno de los hallazgos más importantes de la revisión crítica: al duplicarse la lógica de control de acceso en cuatro servicios casi idénticos, una de las copias quedó incompleta.
- La API REST de BIOPET sigue las restricciones arquitectónicas descritas por **Fielding (2000)** y está correctamente documentada con Swagger UI en su ruta real (`/api/docs`); la ejecución en vivo de cuatro endpoints confirmó que el sistema responde con los códigos de estado, estructuras de datos y cabeceras de seguridad esperados.
- La comparación SOAP vs. REST confirma que la elección de estilo arquitectónico no es una cuestión de preferencia técnica, sino de quién controla el contrato: BIOPET está correctamente construido en REST porque su equipo define el contrato y su consumidor es un frontend propio, mientras que sistemas como el del SRI ecuatoriano siguen exigiendo SOAP por requisitos normativos de interoperabilidad.
- De las tendencias modernas analizadas, PWA es la única que resuelve un problema real y verificable del dominio de BIOPET (conectividad inestable en campo); Jamstack no encaja con un sistema tan dinámico y personalizado por rol, y la IA generativa aportaría más valor en el proceso de desarrollo que dentro del producto.
- Las dos revisiones críticas independientes, pese a partir de evidencia de naturaleza distinta (código fuente vs. experiencia de usuario), convergen en el mismo diagnóstico general: BIOPET está por encima del estándar típico de un PFC de esta asignatura en disciplina de pruebas y documentación de arquitectura, pero no está listo para producción sin corregir al menos un fallo real de autorización y sin cerrar la brecha entre lo que el backend soporta y lo que el

frontend expone.

## 4. Trabajo futuro

1. **Seguridad (corto plazo):** corregir la fuga de datos en el listado de consultas médicas para el rol **DUENO**; rotar y recablear correctamente la clave de la API externa; mover el limitador de intentos de login a Redis; externalizar la contraseña del administrador sembrado.
2. **Paridad funcional:** implementar los componentes Angular de citas y consultas para que el frontend exponga la funcionalidad que el backend ya soporta.
3. **Pruebas:** agregar un test de autorización de listados por rol **DUENO** en consultas médicas (habría detectado la fuga de datos); incorporar pruebas unitarias mínimas en el frontend Angular, que actualmente no tiene ninguna.
4. **PWA:** agregar un *service worker* con estrategia de caché para el listado de mascotas y una cola de sincronización para escrituras pendientes en condiciones de conectividad inestable.
5. **Funcionalidad pendiente ya declarada por el propio equipo** (SRS, sección 2.3): historial clínico completo, telemetría IoT, recomendaciones asistidas, facturación y reportes avanzados.

## Referencias

1. Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. [https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding\\_dissertation.pdf](https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf)
2. Spring. (2024). *Spring Boot reference documentation* (versión 3.2). VMware. <https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/>
3. Pautasso, C., Zimmermann, O., & Leymann, F. (2008). RESTful web services vs. “big” web services: Making the right architectural decision. En *Proceedings of the 17th International Conference on World Wide Web (WWW 2008)*. <https://design.inf.usi.ch/sites/default/files/biblio/www2008-restws-pautasso-zimmermann-leymann.pdf>
4. OASIS. (2004). *Web services security: SOAP message security 1.0 (WS-Security 2004)* [Estándar OASIS]. <http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-soap-message-security-1.0.pdf>
5. springdoc-openapi. (2024). *springdoc-openapi documentation* (versión 2.5). <https://springdoc.org/>
6. PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/16/>
7. Servicio de Rentas Internas del Ecuador. (s.f.). *Ficha técnica: Manual de usuario, catálogo y especificaciones técnicas sobre el proceso de autorización y emisión de documentos electrónicos* (esquema offline, versión 2.26). <https://www.sri.gob.ec/o/sri-portlet-biblioteca-alfresco-internet/descargar/ed555352-46c7-4917-9f61-011b6a9f4600/FICHA%20TE%CC%81CNICA%20COMPROBANTES%20ELECTRO%CC%81NICOS%20ESQUEMA%20OFFLINE%20Versio%CC%81n%202.26.pdf>
8. “Comparative analysis of SOAP and REST APIs” (2024). <https://eprints.unite.edu.mk/1951/1/revista%20-%202024-228-243.pdf>
9. “Performance benchmarking of RESTful and SOAP APIs” (2018). *Journal of Scientific and Engineering Research*. <https://jsaer.com/download/vol-5-iss-11-2018/JSAER2018-05-11-376-390.pdf>
10. “A comparative study of SOAP vs. REST web services provisioning techniques for mobile host”. *ResearchGate*. <https://www.researchgate.net/profile/Dr-K-Wagh/publication/264227921>
11. IETF. (2022). *RFC 9110: HTTP semantics*. <https://www.rfc-editor.org/rfc/rfc9110>

12. W3C. (2007). *SOAP version 1.2 part 1: Messaging framework* (2.<sup>a</sup> ed.). <https://www.w3.org/TR/soap12-part1/>
13. W3C. (2001). *Web Services Description Language (WSDL) 1.1*. <https://www.w3.org/TR/wsdl>
14. Kushki. (s.f.). *API Docs*. <https://api-docs.kushkipagos.com/>

NOTA: El PFC BIOPET, objeto de esta revisión, fue desarrollado por su equipo original (Mariscal Cabrera Jaime Josué, Beltrán Montiel Fred Adrián y Taipe Mora Zaida Melissa). El Equipo B no participó en su implementación, lo que si realiza sobre él es la revisión cruzada, investigación y análisis solicitados para esta actividad.